

TUIA NLP 2026 — Trabajo Práctico Final

Pautas generales

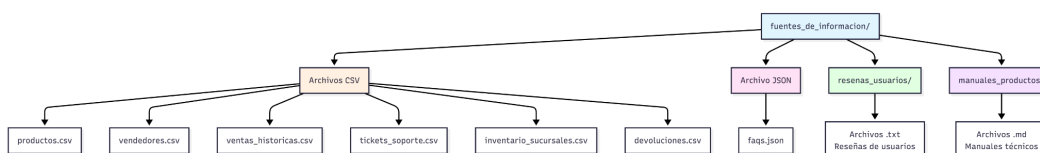
- **Modalidad:** El trabajo deberá ser realizado de forma individual.
- **Entorno:** El proyecto debe desarrollarse íntegramente en un cuaderno de Google Colab.
- **Entregables:** Se debe entregar un informe en PDF y un cuaderno Jupyter (.ipynb) ejecutado y sin errores. Ambos deben estar en un repositorio de Git (GitHub/GitLab) **PRIVADO**, compartido solamente con los integrantes del grupo docente (jpmanson@gmail.com, alan.geary.b@gmail.com)
- **Fecha límite de entrega:** Martes 23/06/2026 23:59:59.

Objetivo del trabajo práctico

Desarrollar un asistente virtual especializado en información de una empresa de electrodomésticos que responda preguntas como:

- Información sobre usos de productos: *"¿Cómo uso mi licuadora para hacer smoothies?"*
- Información sobre precios: *"¿Cuáles son las licuadoras de menos de \$200?"*
- Opiniones de usuarios: *"¿Qué opinan los usuarios de esta cafetera?"*
- Recomendaciones: *"Quiero una licuadora con buenas reseñas"*
- Preguntas frecuentes: *"¿Qué voltaje requiere el rallador digital eléctrico?"*
- Relaciones entre productos: *"¿Qué productos están relacionados con la categoría Cocina?"*

Fuente de información a utilizar



[fuentes de informacion](#)

> **Nota:** Para el primer ejercicio no se utilizarán todos los archivos.

Ejercicio 1 - RAG

1.1 Diseño de las fuentes de datos

A partir del dataset proporcionado, diseñar e implementar **tres repositorios de información** con estructuras y propósitos diferenciados:

- Una **base de datos vectorial**
- Una **base de datos tabular**
- Una **base de datos de grafos**

Para cada una de las tres fuentes se deberá:

- Definir qué información del dataset se almacenará.
- Justificar la decisión explicando por qué ese tipo de almacenamiento resulta adecuado.
- Describir la estructura definida (esquema, modelo de datos u organización elegida).

1.2 Base de datos vectorial

Requisitos obligatorios:

- Implementar un motor de base de datos vectorial ([ChromaDB](#), [FAISS](#), [LanceDB](#), [Milvus Lite](#) o [Qdrant](#)).
- Definir un modelo de embedding que maneje español.
- Definir un `TextSplitter` con parámetros razonables.
- Preparar una interfaz (función) que reciba consulta + filtros y devuelva los K fragmentos más similares.

1.3 Base de datos tabular

Requisitos obligatorios:

- Cargar la información en DataFrames de Pandas o tablas SQL.
- Recopilar información relevante (valores únicos, mínimos/máximos, etc.) para construir un string de formato que el LLM pueda usar.
- Utilizar un modelo de lenguaje con instrucciones de sistema ajustadas (sin pasar todo el dataset) para que genere los filtros necesarios.
- Desarrollar una interfaz (función) que reciba solo los filtros generados por el LLM.

1.4 Base de datos de grafos

Requisitos obligatorios:

- Importar las relaciones desde un DataFrame y prepararlas para inserción.
- Utilizar una base de grafos **local** (sin API externa). La cátedra asignará uno de los siguientes: **Neo4j**, **RedisGraph** o **GrafitoDB**.
- El lenguaje de consulta será **Cypher**.
- Implementar un modelo de lenguaje que convierta lenguaje natural a consulta Cypher.
- Desarrollar una interfaz (función) que reciba la consulta en lenguaje natural y devuelva los resultados.

1.5 Clasificador de Intención Avanzado

Se deben implementar y comparar **dos enfoques**:

1. **Clasificador entrenado** con preguntas sintéticas sobre cada fuente de datos.
2. **Clasificador basado en LLM** con Few-Shot Prompting.

Comparar ambos usando métricas de clasificación y justificar cuál es superior.

1.6 Pipeline de Recuperación (Retrieval)

- **Búsqueda semántica:** Implementar búsqueda híbrida (semántica + BM25) + mecanismo de ReRank.
- **Consultas dinámicas:** Para grafos y datos tabulares **no está permitido** pasar la tabla o el grafo completo al contexto. Se deben generar consultas SQL/Cypher o filtros de Pandas dinámicamente.

1.7 Checkpoint de Recuperación (obligatorio)

Antes de integrar el componente de generación con el LLM, se debe incluir en el cuaderno un **punto de control explícito** del pipeline de recuperación completo.

Para cada uno de los siguientes prompts, el notebook debe mostrar de forma clara:

- Intención detectada por el clasificador y fuente de datos seleccionada.
- Consulta generada (si corresponde) para la fuente tabular o de grafos.
- Fragmentos o registros recuperados **después del re-rank** (contenido, fuente, metadata y puntaje).

Prompts obligatorios de verificación:

1. "¿Cómo uso mi licuadora para hacer smoothies?"
2. "¿Cuáles son las licuadoras de menos de \$200?"
3. "¿Qué opinan los usuarios de esta cafetera?"
4. "Quiero una licuadora con buenas reseñas"
5. "¿Qué productos están relacionados con la categoría Cocina?"

Este checkpoint debe ejecutarse de forma aislada (sin generar respuesta final con el LLM).

1.8 Modelo de Lenguaje (LLM) y Restricción de ejecución local

- **Restricción obligatoria:** Todos los LLMs utilizados en el trabajo (clasificador de intención, generación de consultas Cypher/SQL/filtros, y generación final de respuestas) **deben ejecutarse de forma 100% local** dentro del cuaderno de Colab.

- **No está permitido** el uso de APIs externas (Groq, OpenAI, Anthropic, Together ai, OpenRouter, Cohere, Hugging Face Inference API, etc.) que requieran claves de API.
- Se recomienda el uso de `transformers` u `ollama` con modelos pequeños/instruct (ej: Gemma 4, Phi-3/4, Qwen 2.1, Llama 3.2, etc.) o `llama-cpp-python` con modelos GGUF cuantizados.

1.9 Generación y Conversación

Integrar todos los componentes en un bucle conversacional que:

- Soporte memoria para mantener el contexto.
- Responda en el mismo idioma de la consulta.
- Si no encuentra información relevante, sugiera al usuario reformular la pregunta.

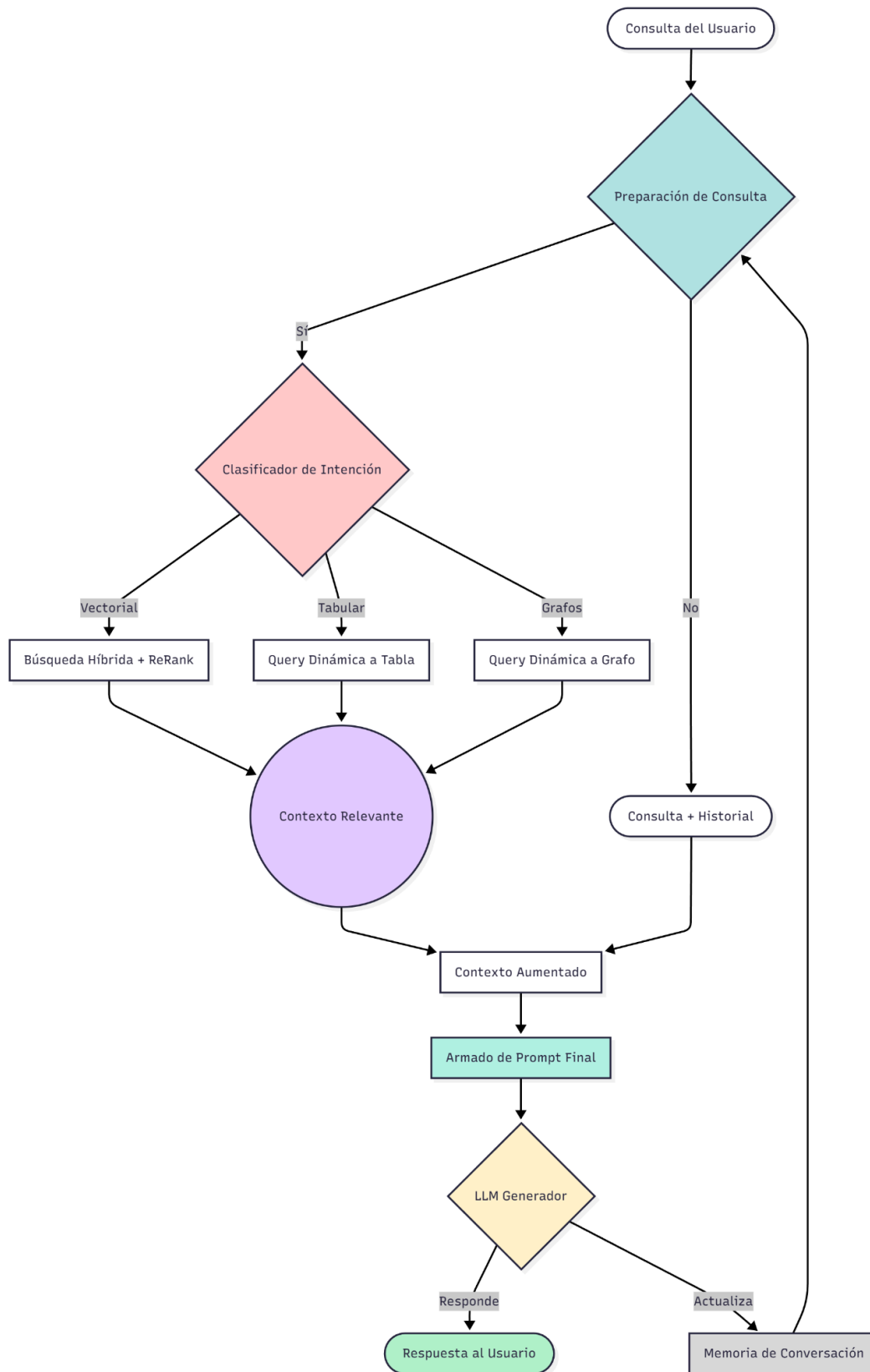


Diagrama de ejemplo del RAG

Ejercicio 2 - Evolución del RAG a un Agente Autónomo

2.1 Requisitos generales

Transformar el sistema RAG en un agente inteligente basado en el paradigma **ReAct** utilizando **LangChain (sin LangGraph)**.

Restricción de LLM local (aplica a todo el TP): El LLM que utilice el agente ReAct también debe ejecutarse de forma completamente local. No se aceptarán notebooks que dependan de APIs externas.

2.2 Creación de Herramientas (Tools)

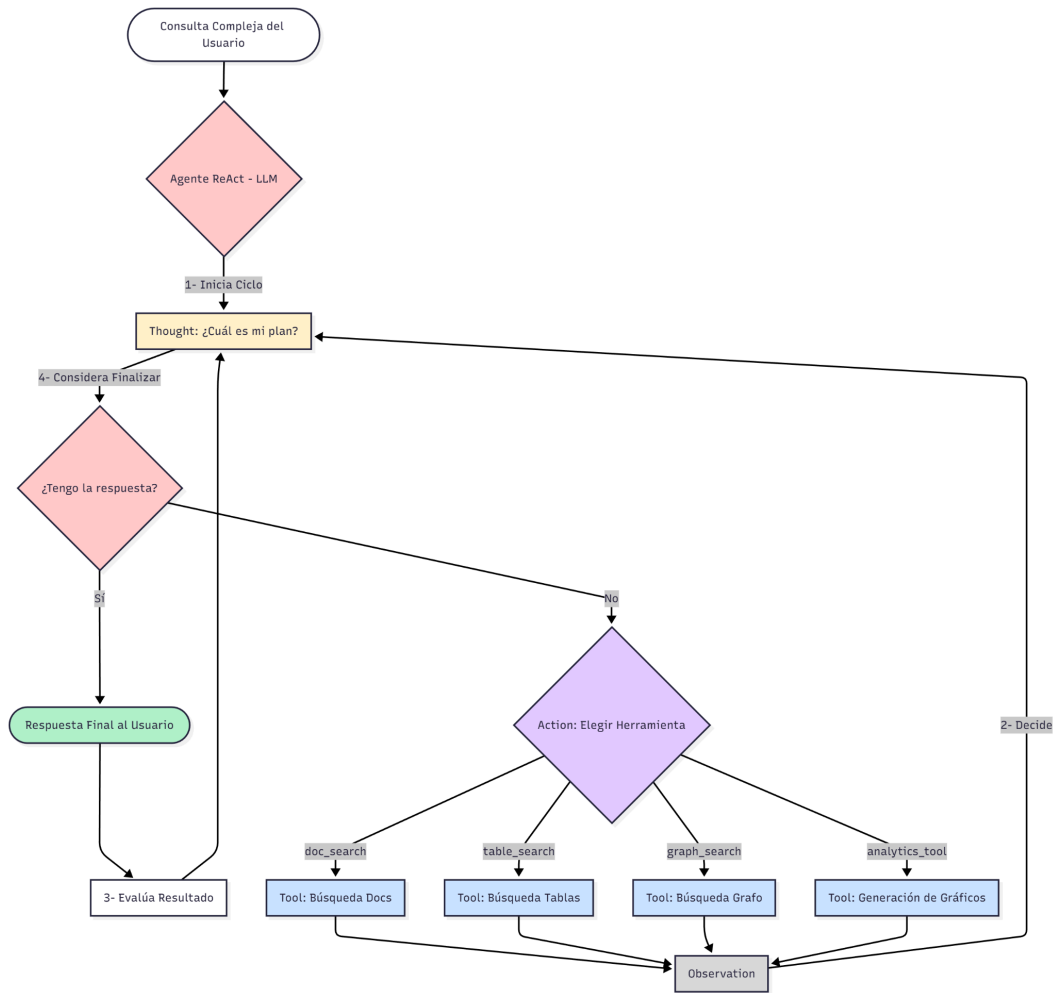
Encapsular la lógica de cada fuente de datos en herramientas independientes:

- ``doc_search()``: Búsqueda en documentos con búsqueda híbrida y re-rank.
- ``table_search()``: Consultas dinámicas a datos tabulares.
- ``graph_search()``: Consultas dinámicas a la base de datos de grafos.
- ``analytics_tool()``: Búsqueda en base de datos SQL + generación de gráficos con matplotlib (ejemplo: distribución de medios de pago).

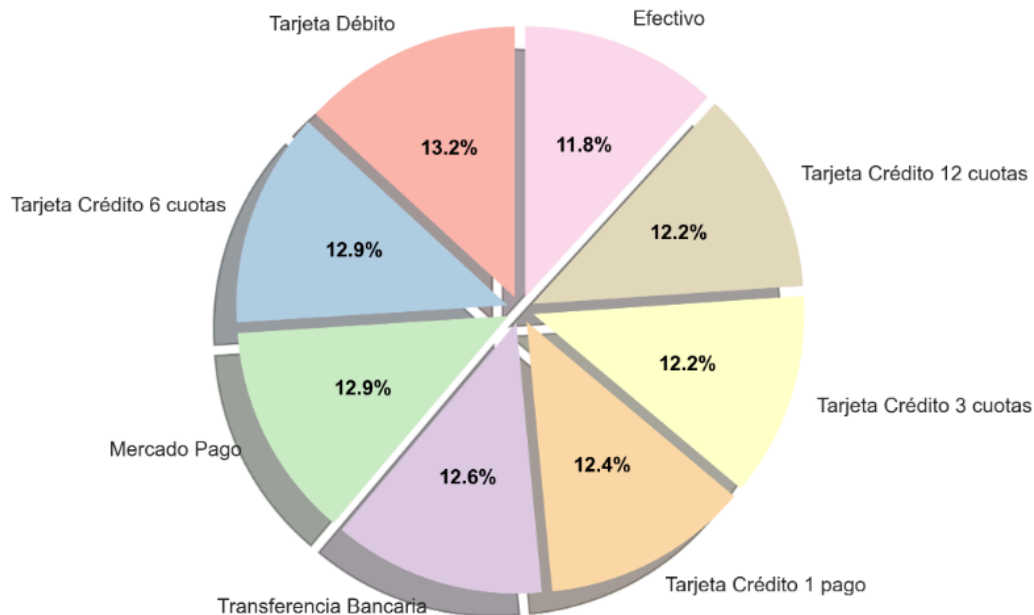
2.3 Prompt del sistema del agente

Construir un prompt de sistema cuidadosamente diseñado que indique al agente:

- El propósito de cada herramienta.
- Cómo debe razonar (Thought) para elegir una o varias herramientas (Action).
- Cómo construir una respuesta completa.



Distribución de Métodos de Pago (por transacciones)



Diagramas de ejemplo del Agente ReAct

Presentación del informe

El informe debe estar en formato **PDF** y contener:

- Justificaciones de las decisiones tomadas respecto a:
 - Modelo de embedding
 - Text Splitter
 - Clasificador de intención
 - Modelo de lenguaje (local)
- Resultados de ejecución de ambos ejercicios (mínimo **5 pruebas** por ejercicio) evaluando todo el potencial del chatbot.
- Análisis de mejoras posibles para perfeccionar el rendimiento del agente y solucionar fallos detectados.
- Mínimo de 15 páginas
- Bibliografía adicional o referencias externas (si corresponde)